Agentic Engineering

Patterns that make AI coding tools actually work

Press Space to advance

Agenda

1. **What is an agentic coding tool?** — beyond autocomplete
2. **Tool landscape** — chat, IDE agent, CLI agent — where each fits
3. **The agentic loop** — how the model reasons and acts
4. **Context engineering** — giving the model what it needs, AGENTS.md, skills
5. **Real examples: good prompts** — patterns that work
6. **Anti-patterns** — what goes wrong and why
7. **Techniques** — plan mode, error context, iterative narrowing
8. **The shift in how we work** — objective-based, review strategy, validation loops, feedback signals, context curation, MCP hygiene
9. **The future: always-on agents** — autonomous workloads and the questions they raise
10. **Takeaways** — a checklist you can use today

Autocomplete vs. Agent

Agentic Tool

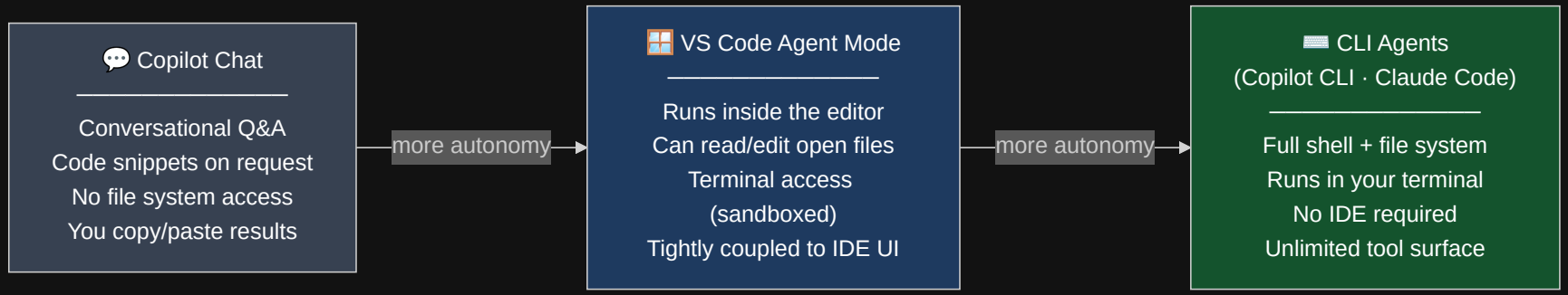
- Given a goal, plans and executes
- Calls tools: reads files, runs shell, searches
- Iterates until done — or asks when stuck

```
"Migrate the Dagster deployment to Helm.
 The current config is at src/.../__main__.py"
```

You describe the outcome. The agent figures out the steps.

The Spectrum of AI Coding Tools

Not all AI tools are the same — they sit at different points on an autonomy axis.



Key distinction: Chat tools are *reactive* — you ask, they answer. Agentic tools are *proactive* — you give a goal, they plan and execute.

Tool Comparison

💬 Copilot Chat · *inline / sidebar*

✅ Strengths

- Fast Q&A on code
- Explain / document / review
- Low risk — no side effects

❌ Limitations

- No file system access
- You apply suggestions manually
- Context limited to open files
- No shell execution

Best for: Quick edits, code review prep, explaining unfamiliar code

🪟 VS Code Agent Mode · *in-editor*

✅ Strengths

- Stays in your IDE workflow
- Can edit multiple files
- Integrated terminal access
- Sees your workspace tree

❌ Limitations

- Tightly coupled to VS Code
- Approval flow is per-file
- Harder to script or automate
- Context from open tabs only

Best for: Contained feature work, single-repo changes in VS Code

⌨️ Copilot CLI / Claude Code · *terminal*

✅ Strengths

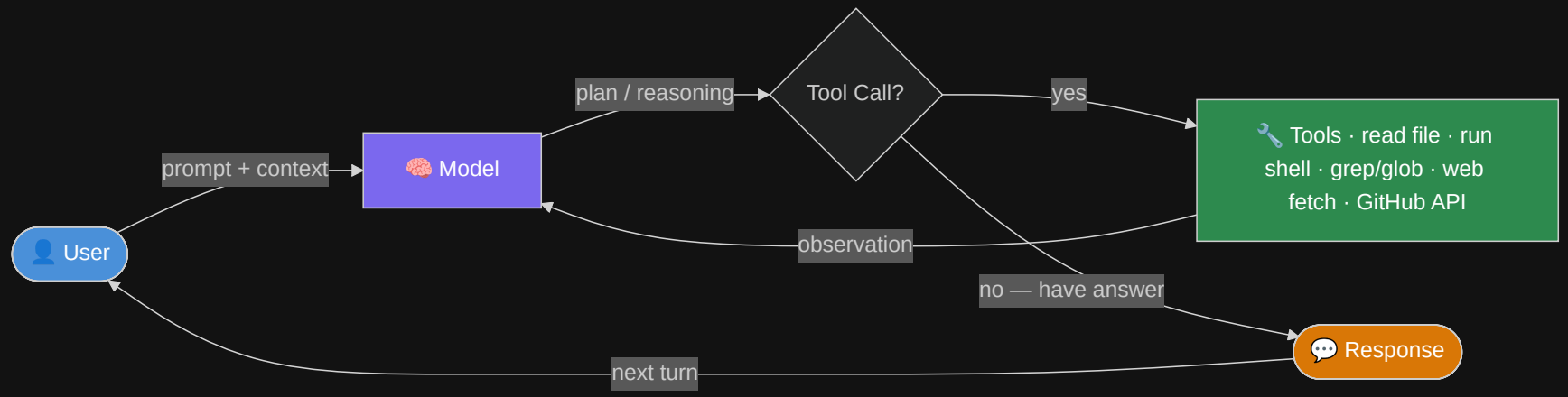
- Full shell & filesystem
- Works across any editor
- Scriptable, composable
- Multi-repo, multi-dir tasks
- Plan mode + approval flow

❌ Limitations

- No visual IDE integration
- Requires terminal comfort
- Broader blast radius

Best for: Migrations, refactors, multi-file changes, infra tasks

The Agentic Loop



Each turn: model reads context → decides to call a tool or respond → result feeds back in → repeat

What the Agent "Sees"

The model reasons over everything in its **context window** each turn:

- 📄 **Your prompt** — the instructions you write
- 🗂️ **File contents** — whatever it has read via tools
- 🖥️ **Shell output** — results of commands it ran
- 🧠 **Its own prior reasoning** — tool calls and observations
- 📋 **System instructions** — tool definitions, behavior guidelines
- 🌐 **External content** — web pages, GitHub API responses, docs

Implication: The agent can only act on what's in context. If the relevant file path, error, or constraint isn't in the prompt — the model will guess, hallucinate, or ask.

Copilot CLI Modes

💬 Interactive

Turn-by-turn conversation. You guide each step. Good for exploration and debugging.

🤖 Autopilot

Agent executes autonomously, auto-approving changes. Best for well-scoped, trusted tasks.

📋 Plan Mode

`[[PLAN]]`

Agent plans first, presents for review, then executes. Best for complex or risky changes.

Rule of thumb: Use plan mode when the blast radius is large or the approach isn't obvious. Use autopilot when the task is scoped and you trust the direction.

Context Engineering

Giving the model the right information to make the right decision

The Anatomy of a Good Prompt

Every effective agentic prompt answers these questions:

- **What exists now?** Current state, relevant files, tech stack
- **What should change?** The desired outcome
- **Where is it?** Specific file paths, directories, modules
- **What are the constraints?** Backward compat, style, scope
- **How do I know it worked?** Success criteria or test command

Current state

```
I currently have X that does Y.
The relevant code is at path/to/file.py.

[Desired outcome]
I would like to change it to do Z.

[Constraints]
Existing usages throughout the repo
should continue to work.

[Success criteria]
Running `pytest` should pass.
```

File Paths Are Not Optional

The agent reads your codebase by calling tools. It doesn't "know" your project layout.

❌ Vague

```
Update the vault authentication module
to support OIDC.
```

Agent must guess which file. May read the wrong one, ask a clarifying question, or hallucinate a path.

✅ Specific

```
Update the
packages/ol-orchestrate-lib/src/
ol_orchestrate/resources/secrets/vault.py
module to support OIDC as an
authentication method, taking a role
as an optional parameter.
```

Agent goes directly to the right file. No guessing.

State What Exists, Then What Should Change

Give the agent a narrative: here's where we are, here's where we're going.

> "The `dg_projects` directory contains self-contained Dagster code locations that work with the `dg` CLI. These were previously all located in a single shared location under `src/ol_orchestrate`. The shared logic is now located in `packages/ol-orchestrate-lib`. Review the contents of `ol-orchestrate-lib` and the imports in the `dg_projects` code locations. For any code which exists in `ol-orchestrate-lib` that is only used by a single code location, move that code into the code location directly."

📍 **History**
Where it came from

🗂️ **Locations**
Specific directories

🎯 **Action**
Precise instruction

Include Constraints

Agents are optimistic — they'll make sweeping changes if you don't set limits.

❌ No constraints

```
Update OLEKSAuthBinding to support
multiple service accounts.
```

Agent may break existing call sites, change public API shape, or refactor things you didn't ask for.

✅ With constraints

```
Update it to allow passing multiple
service accounts through, while making
sure that existing uses of it and
dependent components are still
functional in their other usages
throughout the repository.
```

Common constraints worth stating: backward compatibility, performance bounds, file scope, style conventions, what *not* to touch.

Reference External Docs

Agents can fetch URLs. Use this for fast-moving libraries, proprietary APIs, or anything post-training.

```
I would like to migrate this to use
the Dagster Helm chart for deployment
on Kubernetes. The documentation page
that covers this is at
https://docs.dagster.io/deployment/
guides/kubernetes/deploying-with-helm
```

- `llms.txt` — Many projects publish a flat doc index optimized for LLM context
- **API reference URLs** — For SDKs the model may not know well
- **GitHub issues / PRs** — Link the ticket that describes the desired behavior
- **Internal runbooks** — Paste or link relevant ops docs

Include Success Criteria

Tell the agent how you'll know it worked. This guides its verification behavior.

Explicit test command:

```
Evaluate the dependencies in the
pyproject.toml files for each project
in dg_projects to ensure that they are
properly reflected and will result in
a working docker image.
```

Accept / reject guidance:

```
Run `uv run mypy dg_projects` to
evaluate errors and propose fixes.
Valid fixes can also include ignoring
errors if they do not prevent the
functionality of the logic.
```

Success criteria also help the agent decide *when to stop* — without them, it may over-engineer or quit too early.

The Context Budget

Context windows are large but not infinite. Quality > quantity.

Include:

- The specific files you want changed
- The error or behavior you're seeing
- Related config files or schemas
- Constraints and success criteria

Exclude:

- Unrelated parts of the codebase
- Entire test suites "just in case"
- Repeated context already established
- Boilerplate the agent can find itself

The focused prompt wins:

```
# Instead of:
"Here is my entire repo. Fix the auth bug."

# Use:
"The Vault authentication in
packages/.../vault.py is failing with
OIDC in production. The relevant code
is in the `authenticate()` method.
Update it to support the OIDC flow
using the hvac library's OIDC API."
```

AGENTS.md — Base Context That's Always There

Instead of re-explaining your project every session, codify it once.

What `AGENTS.md` is:

An `AGENTS.md` file at the root of your repository is automatically loaded by supported agents at the start of every session. It's your standing brief — the context the agent always needs, so you never have to repeat it.

What to put in it:

- Project layout and key directories
- How to run tests, lint, and build
- Coding conventions and style rules
- What tools are available (`uv` , `dg` , `helm` , etc.)
- Things the agent should *never* do (force-push, drop tables, etc.)
- Links to internal docs or architecture references

```
# AGENTS.md

## Project layout
- `dg_projects/` — Dagster code locations (one per
- `packages/ol-orchestrate-lib/` — shared resource
- `src/ol_dbt/` — dbt project

## Running things
- Tests: `uv run pytest`
- Type check: `uv run mypy dg_projects packages`
- Build image: `docker build -f <dir>/Dockerfile .

## Conventions
- All Dagster resources use HVAC for Vault auth
- Kubernetes auth in non-dev environments
- Never modify `pyproject.toml` lock files manuall

## Do not
- Push directly to main
- Modify shared resources without checking all con
```

Agent Skills — Incidental Context on Demand

Skills extend the agent with specialized, task-specific knowledge — loaded only when relevant.

What skills are:

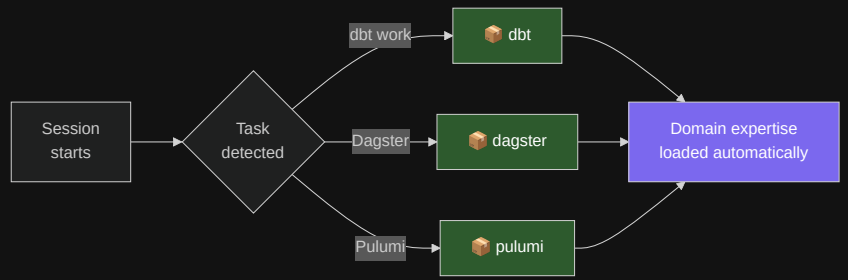
- Installable packages that add domain knowledge and tool guidance
- Activated when the agent detects a matching task
- Discoverable at **skills.sh** — a public registry of community and vendor skills

Examples of available skills:

- `dagster` — project structure, `dg` CLI patterns, asset definitions
- `dbt` — model building, semantic layer, unit tests
- `pulumi` — component authoring, output handling, best practices
- `flutter`, `superset`, `dagster-integrations` — tool-specific expertise

Skills inject the right domain knowledge without you explaining it — and aren't loaded when irrelevant, so they don't pollute unrelated sessions.

Agent Skills — How It Works



📄 AGENTS.md

Base context — always loaded. Project layout, conventions, tooling, do-nots.

📦 Skills

Incidental context — loaded on demand. Domain expertise for the current task only.

Together they eliminate most of the orientation boilerplate you'd otherwise write at the start of every session.

Real Examples

Good Prompts from the Field

Example: Context-Rich Migration

> "I currently have a deployment of Dagster that uses docker-compose on an EC2 server. This is deployed using Pulumi. The deployment logic is located at `src/ol_infrastructure/applications/dagster/__main__.py` and the AMI build logic is located at `src/bilder/images/dagster`. I would like to migrate this to use the Dagster Helm chart for deployment on Kubernetes. The documentation page that covers this is at `https://docs.dagster.io/deployment/guides/kubernetes/deploying-with-helm`."

🏗️ **Current state**
docker-compose on EC2 via Pulumi

🗂️ **File locations**
Both the deployment and image build paths

🎯 **Goal**
Helm chart on Kubernetes

📚 **Docs**
Direct URL to the relevant guide

Example: Scoped Module Update

> "Update the `packages/ol-orchestrate-lib/src/ol_orchestrate/resources/secrets/vault.py` module to support OIDC as an authentication method, taking a role as an optional parameter."

Why it works:

- Exact file path — no searching required
- Single, clear behavior change
- Interface detail specified (`role` as optional param)
- Implies "don't break existing auth methods" by saying "support OIDC *as well*"

Pattern: `Update <exact path> to <specific behavior>, taking <parameter> as <type>.`

Simple tasks don't need long prompts — they need *precise* ones.

Example: Refactor with History

"This repository consists of a Dagster project located under `src/ol_orchestrate` and a dbt project located at `src/ol_dbt`. I would like to refactor the Dagster project to follow the new `dg` style workspace and project layout. There are many shared libraries and resources in the Dagster project that are used across different 'projects' or 'code locations'. I would like to restructure the project to make logically separate `dg` projects that correspond to: lakehouse, open edX, edxorg, legacy openedx, canvas. The shared code should be extracted into a library package."

📖 **History**
What the repo structure was before

🗂️ **Topology**
Named code locations = named projects

📦 **Outcome**
Library extraction strategy

What Made These Work?

Structural patterns:

- Open with the current state in one sentence
- Name every relevant file path explicitly
- State the goal in terms of behavior, not implementation
- Include constraints inline, not as afterthoughts
- Give the agent a URL when docs exist
- Define "done" — test command or acceptance criteria

The mental model:

> Imagine you're writing a ticket for a new engineer joining the team today — no context, no prior history, no Slack thread to read.
> That's what you're writing for the agent.

Every session starts fresh. The agent has no memory of yesterday's conversation.

Anti-Patterns

What goes wrong and why

Anti-Pattern: The Raw Error Dump

❌ What gets sent

```
dagster._core.errors.DagsterInvalidDefinitionErro
resource with key 'openedx' required by op
'openedx__courseware' was not provided.
Please provide a ResourceDefinition to key
'openedx', or change the required key to one
of the following keys which points to an
ResourceDefinition: ['io_manager',
's3file_io_manager', 'vault', 's3',
'duckdb', 'openedx_mitx', ...]

  File "/app/.venv/.../definitions.py", line 847
  ... [14,000 more characters of stack trace]
```

What the agent is missing:

- What you were trying to do
- What you already tried
- Which approach you want taken
- Whether this is a config issue, code issue, or both

What happens instead:

- Agent picks the most obvious fix (may be wrong)
- Spends turns investigating context it needed upfront
- May ask clarifying questions — which you could have answered in the prompt

Anti-Pattern: The Raw Error Dump — Fixed

"The openedx code location is failing to load with a `DagsterInvalidDefinitionError` — the `openedx` resource key is missing. The definitions are in `dg_projects/openedx/definitions.py`. The resource was previously injected at the job level but we've since moved to repository-level resource binding. Update the definitions to bind the `openedx` resource at the repository level using the existing resource definitions in `packages/ol-orchestrate-lib/src/ol_orchestrate/resources/`."

🔍 **Error summary**
One line, not 14k chars

🗂️ **Context**
What changed that caused this

🎯 **Direction**
Which approach to take

Anti-Pattern: Context-Free Follow-Ups

Each new message is a **continuation of the same context window** — but only if that context is still there.

❌ Loses context

```
> Continue fixing the mypy errors
```

```
> Migrate the next code location
```

```
> Migrate the legacy_openedx code
  location next
```

```
> Perform the migration now
```

The agent has to guess which errors, which location, what migration pattern to use.

✅ Re-anchors context

```
> There are type errors in dg_projects.
  Run `uv run mypy dg_projects packages`
  to get the full list and fix them.
```

```
> Migrate the canvas code location next
  using the same pattern you used for
  legacy_openedx: separate pyproject.toml,
  Dockerfile, and build.yaml.
```

Anti-Pattern: Assumed Context

New sessions start with a blank slate.

❌ What was sent

```
Defaulting to `k8s-data`
```

This was the entire prompt — a fragment from a previous conversation that no longer existed in context.

The agent had no idea what this referred to.

💡 What to do instead

Before starting a new session on an in-progress task, provide a quick orientation:

```
We're continuing the Dagster Helm
migration from yesterday. The current
state: all code locations have been
migrated except canvas. The deployment
config is at src/ol_infrastructure/
applications/dagster/__main__.py.
Next: migrate the canvas code location.
```

Anti-Pattern: Rubber-Stamp Approval

Plan mode asks for review. Approving without reading removes the benefit entirely.

❌ Dangerous

```
> Yes, proceed with all changes.
```

```
> Looks good, go ahead.
```

If the plan has a scope error, wrong approach, or missed constraint — you’ve approved it without catching it.

✅ Engage with the plan

Things to check before approving:

- Does the file list match what you expected?
- Is the approach what you had in mind?
- Are any unrelated files in the change set?
- Does the plan account for backward compat?
- Does it match your constraint from the prompt?

Plan mode exists to catch mistakes *before* they're made. Use it.

Anti-Pattern: Ambiguous Scope

Vague scope forces the agent to make assumptions — and it will.

❌

> "I have migrated the deployment of these Dagster definitions to run on Kubernetes. Update all of the handling of Vault authentication for non-dev environments to use Kubernetes auth."

What's missing:

- Which directories / modules contain the auth handling?
- How many call sites are there?
- What does "Kubernetes auth" mean here — OIDC? Service account tokens? HVAC?
- Should non-Kubernetes environments (local dev) be unchanged?
- What existing auth methods should be preserved?
- Is this a config change, code change, or both?

Anti-Pattern Gallery

Pattern	Example	Why it fails
Raw error dump	14k stack trace, no preamble	Agent picks wrong fix, wastes turns
Context-free follow-up	"Continue fixing the mypy errors"	Agent guesses which errors, which file
Assumed context	`"Defaulting to k8s-data"`	Agent has no idea what this refers to
Rubber-stamp	"Yes, proceed with all changes"	Bypasses plan review entirely
Ambiguous scope	"Update all Vault auth to use k8s"	Agent makes sweeping, wrong assumptions
Typo/accident	`"ghc"`	Wastes a full agent turn
No constraints	"Migrate this to the new API"	Breaks existing consumers

Patterns & Techniques

Plan Mode

Use `[[PLAN]]` at the start of your prompt for complex or risky tasks.

```
[[PLAN]] I want to migrate the Dagster deployment from docker-compose to Helm.
The current infra is at src/ol_infrastructure/applications/dagster/__main__.py.
The target is a Kubernetes cluster. Documentation at https://docs.dagster.io/...
```

📋 **Agent plans first**
Lists files to change, approach, decisions

👀 **You review**
Catch scope errors before code is written

✅ **Then executes**
With your approval, carries out the plan

When to use: Large refactors, migrations, changes touching many files, anything where the wrong approach would be expensive to undo.

Iterative Narrowing

Start broad to orient the agent, then narrow to targeted actions — re-anchoring each turn.

Turn 1 — Broad exploration:

```
Review the contents of ol-orchestrate-lib and the imports in the dg_projects
code locations. For any code that exists in ol-orchestrate-lib and is only used
by a single code location, identify it and tell me what you find before making
any changes.
```

Turn 2 — Targeted action:

```
Move the identified single-use modules into their respective code locations.
Start with the openedx code location.
```

The agent explored first and reported back. Turn 2 is precise because Turn 1 already established shared understanding — no guessing required.

Iterative Narrowing — The Pattern

The four-step loop:

1. **Explore** — confirm the agent's understanding before acting
2. **Scope** — agree on exactly what will change
3. **Execute** — one logical unit at a time
4. **Verify** — check the result, then continue

Why it matters:

Agents can **drift** on large tasks — small misunderstandings early compound into large corrections later.

Checkpoints give you an off-ramp **before** the agent has done significant work in the wrong direction.

Each verified unit also **anchors** the next turn with confirmed context.

The Error Context Pattern

When something breaks, give the agent three things: error, context, direction.

Error (summary): The canvas code location is failing to start with a Vault authentication error — it's attempting GitHub auth instead of Kubernetes auth.

Context: We recently migrated this deployment to Kubernetes. The other code locations in `dg_projects/` were updated to use the HVAC Kubernetes auth method in an earlier session. Canvas was missed.

Direction: Update `dg_projects/canvas/resources/vault.py` to use Kubernetes auth using the same pattern as `dg_projects/openedx/resources/vault.py`.

⚠️ **Error (summary)**
One sentence — not a full trace

📖 **Context**
What changed, why it matters

🧭 **Direction**
Explicit "use this pattern"

Tool Awareness

Agents have tools. Reference them in your prompt to guide behavior.

Shell / CLI tools:

```
Run `uv run mypy dg_projects packages`
to get the full list of errors.
```

File search:

```
Review all pyproject.toml files
in the dg_projects directory.
```

Web fetch:

```
The documentation is at
https://docs.dagster.io/llms.txt
```

GitHub integration:

```
Open a technical issue in this
repository to track processing
course XML from open edX courses.
```

Verification:

```
After making changes, run
`docker build .` to verify the
image builds correctly.
```

You don't need to specify every tool — the agent chooses them. But naming a command or URL signals *how* to verify and where to look.

Model-Agnostic Principles

These patterns work across Claude, GPT, Gemini, and whatever comes next.

🗂️ **Specificity beats cleverness**
Exact file paths > clever descriptions

📖 **History enables continuity**
Summarize prior sessions at the start of new ones

🎯 **Constraints prevent drift**
Say what not to touch, not just what to change

✅ **Define done**
Test command > vague "make it work"

🔍 **Review before you approve**
Plan mode is only useful if you read the plan

📦 **One concern per prompt**
Mixed goals produce mixed results

The Shift in How We Work

From writing code to directing agents

From Task-Based to Objective-Based Work

Agentic tools don’t just make you faster at the same tasks — they change which tasks are yours.

Before: Task-Based

You own the implementation details.

- Write the migration function
- Wire up the test fixture
- Update the import paths
- Fix the type error on line 47

Mental model: *"What do I need to type next?"*

After: Objective-Based

You own the outcome and the judgment calls.

- Define the migration strategy
- Set the acceptance criteria
- Identify the constraints
- Review the diff and approve

Mental model: *"Is this the right outcome?"*

The skill that grows in value: **knowing what good looks like** — not knowing how to type it.

The New Role: Agent Director

The engineer's job shifts up the abstraction stack.

🗺️ Decompose problems

Break large goals into well-scoped, sequenced agent tasks — small enough to verify, large enough to be worthwhile

📋 Write the brief, not the code

A well-written prompt with clear context and constraints is the deliverable — the agent writes the code

🔍 Review with intent

The review is where your engineering judgment lands. You're verifying correctness, not authoring it

🧭 Set the architecture

Agents implement within the boundaries you set. Patterns, abstractions, and system design remain human decisions

⚠️ Manage risk

Decide what needs a plan, what gets reviewed, what can ship directly — and own the consequences

🔁 Iterate fast

Multiple agent sessions per hour is normal. The bottleneck moves from writing to reviewing

🤔 Open question: commit the plan?

Agents generate `plan.md` files during complex sessions. Should these be committed as a historical artifact — documenting *why* a change was made alongside the code that made it? Or are they noise in the git log?

Committing plans gives future engineers (and agents) architectural context. Excluding them keeps history clean. Your team should decide deliberately.

🌿 Scaling up: git worktrees

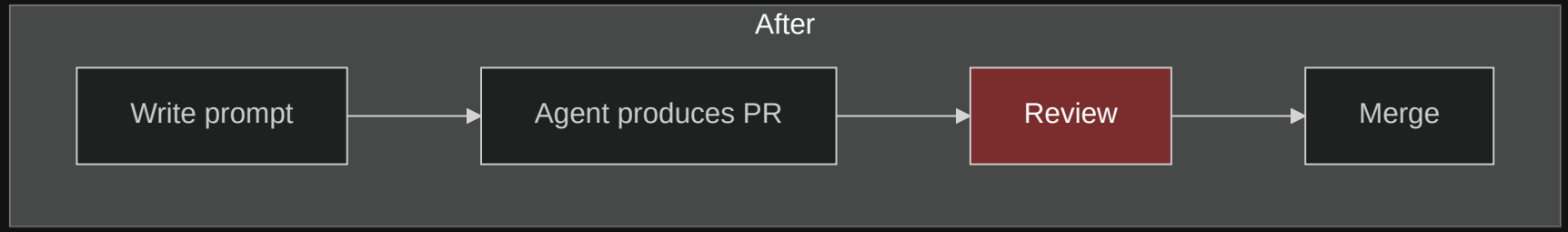
Running multiple agent sessions in parallel requires isolated working directories. `git worktree` lets you check out separate branches into separate directories from a single repo — each agent gets its own tree with no interference.

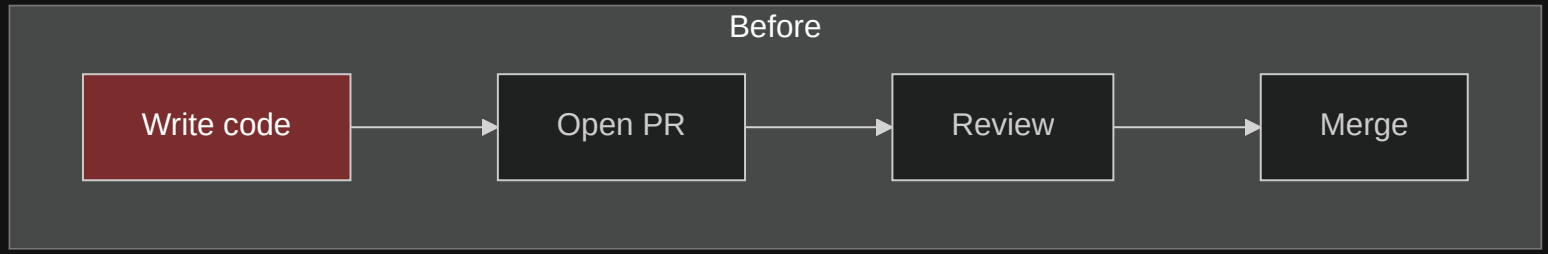
```
git worktree add ../feature-x feature/x
```

One agent refactors auth while another fixes the CI pipeline — no stash conflicts, no branch switching.

The Bottleneck Shifts to Review

When agents can produce a PR in minutes, the queue that backs up is not implementation — it's review.





The Bottleneck Shifts to Review

What this means for teams:

- PR volume increases dramatically
- Human review time is the scarce resource
- Waiting for review becomes the velocity limiter
- Batch review strategies start to matter

Two responses:

1. **Automate first-pass review** — let tools do the initial triage before a human looks
2. **Change review strategy** — not every PR needs the same level of scrutiny

Automated First-Pass Review

Use AI tooling to clear the noise before a human reads a single line.

🤖 GitHub Copilot Reviewer

Inline PR review in GitHub. Flags code quality issues, suggests improvements, explains diffs. Runs as part of the PR workflow — a review is posted automatically when a PR opens.

Best for: Style, logic errors, missing test coverage, documentation gaps

🔷 Gemini Code Review

Google's Gemini model integrated into the PR review flow. Focuses on correctness, security antipatterns, and architectural coherence. Can be configured per-repo with custom guidelines.

Best for: Security review, complex logic validation, cross-cutting concerns

🛡️ Sentry Seer

AI-assisted issue triage and root cause analysis. Surfaces production errors likely introduced by a given PR, correlates stack traces to the diff, and suggests fixes.

Best for: Connecting PRs to production impact, catching regressions before they escalate

The goal: by the time a human reviewer opens the PR, the trivial issues are already resolved and the automated tools have flagged anything

Ship, Show, Ask

Not every change needs the same review. Categorize by risk.

🚀 Ship

Merge directly to main. No pre-merge review required. CI is green, author is a trusted project owner.

Examples:

- Documentation updates
- Adding tests to existing features
- Minor bug fixes with green CI
- Dependency version bumps (automated)

👀 Show

Merge, then review. High velocity, maintained visibility. Review happens post-merge for knowledge sharing.

Examples:

- Standard feature work
- Non-critical refactors
- Tooling and DX improvements

🙋 Ask

Traditional PR. Review required before merge. Reserved for high-stakes or uncertain changes.

Examples:

- Complex refactors
- Security-critical code
- Architectural changes
- Changes the author is unsure about

With agents producing code at high throughput, *Ship* and *Show* are where you reclaim velocity. *Ask* is where your judgment still gates the outcome.

Validation & Feedback Signals

The agent course-corrects from feedback. Give it real signals, not just approval.

❌ Weak feedback loop

```
> Looks good, make the same change
  to the other modules.
```

```
> I think that should work, go ahead
  and commit it.
```

The agent proceeds on your vibe check. If something was subtly wrong, it propagates.

✅ Signal-driven feedback

```
> Run `uv run pytest dg_projects/canvas`
  and verify the tests pass before
  moving to the next module.
```

```
> The type errors are fixed but the
  docker build is still failing with:
  [error output here]
  Investigate and fix.
```

Objective signals (test pass/fail, build output, type errors, linter output) are more reliable than subjective approval. Build verification into the prompt, not as an afterthought.

Validation Loops — Telling the Agent When It’s Done

The most reliable "done" signal is a tool that can’t be argued with.

🧪 Tests

The gold standard. A passing test suite is an unambiguous signal.

```
Run `uv run pytest dg_projects/`
and verify all tests pass before
committing.
```

🔍 Type Checkers & Linters

Catch regressions the agent can't see without running.

```
After each change, run
`uv run mypy dg_projects packages`
and fix any new errors before moving
on.
```

🔨 Build & Package Checks

Verify the artifact actually works, not just the source.

```
Run `docker build .` to verify
the image builds before marking
complete.
```

🪝 Pre-commit Hooks

Run the same checks humans run — automatically, before commit.

```
Run `pre-commit run --all-files`,
fix any
failures, then commit with the
message:
"feat: add OIDC auth to vault
resource"
```

Let the agent commit — a well-formed message is part of the deliverable.

⚙️ CI Simulation

Run what CI will run locally so the PR is clean before it opens.

```
Run `make ci` (or equivalent)
to simulate the full CI pipeline
locally before pushing.
```

📋 Acceptance Criteria

When there's no automated check, define explicit conditions.

```
The task is complete when:
- All existing tests still pass
- The new endpoint returns 200
- No new mypy errors are introduced
```

Information Curation

Context quality degrades as it grows. The best context is focused, not exhaustive.

Curate what goes in:

- Reference only the files relevant to the task
- Link documentation pages, not entire doc sites
- Paste the specific error — not the full 14k-char trace
- Use `@file` tagging to explicitly scope context
- Summarize prior session state in 3–5 sentences

Signs your context is bloated:

- Agent asks questions you already answered
- Agent repeats work it already did
- Responses get slower and less precise
- Agent explores files clearly unrelated to the task

The curation principle:

> Every token in context should be earning its place.

If a file, error message, or background detail isn't directly relevant to the current task, leave it out. You can always add context in a follow-up turn — you can't un-pollute a context window.

Think of context like a good bug report: just enough to reproduce the problem, nothing more.

Tool Overload — The MCP Problem

More tools ≠ more capable. Irrelevant tools actively degrade agent performance.

How MCP servers affect agents:

- Each MCP server adds tool definitions to context
- Tool definitions consume tokens *every turn*
- The model considers all available tools when deciding what to call
- Irrelevant tools create **noise in the reasoning layer**

What goes wrong with too many tools:

- Agent calls a Sentry tool when doing a Python refactor
- Agent explores Slack threads instead of reading the codebase
- Tool descriptions crowd out relevant file content
- More options → slower, less focused decisions

MCP hygiene:

```
# Don't load this for a code refactor:
- Sentry MCP
- Slack MCP
- Linear MCP
- Jira MCP
- Datadog MCP

# Do load this for a code refactor:
- GitHub MCP (for PR/issue context)
- Filesystem tools (built-in)
```

⚠️ Configure MCP servers per-task or per-project, not globally. A Copilot CLI session for infra work should not have your analytics tooling loaded.

The Future: Always-On Agents

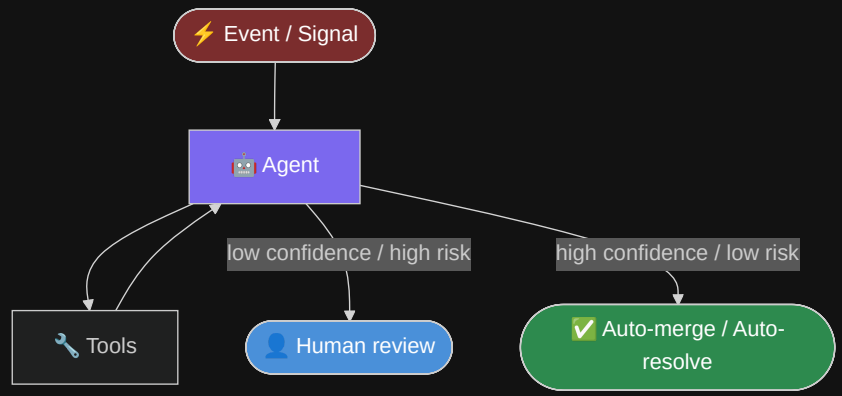
From tools you invoke to workloads that run continuously

Beyond the Prompt: Autonomous Agents

Today's pattern: human writes prompt → agent executes → human reviews. The next pattern: agent watches for signals and acts without being asked.

The shift:

- Agents move from **request/response** to **event-driven**
- Triggers replace prompts: a Sentry alert, a failed deploy, a new CVE
- The human role shifts further — from directing tasks to setting policy
- "What should the agent do when X happens?" becomes the design question



Autonomous Agentic Workloads

What always-on agents could look like in practice:

🛡️ Sentry Auto-Remediation

A new issue fires in Sentry. The agent reads the stack trace, locates the relevant code, opens a PR with a fix, runs tests, and — if CI is green and confidence is high — merges under the *Ship* policy.

Signal: Sentry alert · **Output:** Fix PR or escalation

📦 Dependency & CVE Response

A CVE is published against a dependency. The agent evaluates impact, upgrades the package, runs the test suite, and raises a PR. Severity determines whether it *Ships* automatically or goes to *Ask*.

Signal: Dependabot / CVE feed · **Output:** Upgrade PR

📉 Infrastructure Optimization

An agent monitors spend and performance metrics. When it detects over-provisioned resources or unused infrastructure, it proposes or applies changes within pre-approved bounds.

Signal: Cost/perf metrics · **Output:** Infra PR or Pulumi apply

🧪 Test Gap Detection

After every merge, an agent analyzes coverage delta and code churn. It identifies under-tested paths and opens PRs adding tests — treating coverage as a continuous process.

Signal: Post-merge CI · **Output:** Test coverage PRs

📋 Issue Triage & Enrichment

Incoming GitHub issues are automatically labeled, reproduced in a sandbox, linked to relevant code, and enriched with context before a human ever reads them.

Signal: New GitHub issue · **Output:** Labels, reproduction, code links

📈 Performance Regression Response

After each deploy, an agent compares key metrics against baseline. On regression, it bisects recent commits, identifies the likely culprit, and reverts or opens a fix PR.

Signal: Post-deploy metrics · **Output:** Revert or fix PR

The Design Questions That Matter Now

Autonomous agents raise questions your team should answer *before* you build them.

🔐 Authority boundaries

What is the agent allowed to do without human approval? Which actions require review regardless of confidence? Where does the blast radius become unacceptable?

Define policy, not just capability.

📊 Confidence and escalation

How does the agent know when to act vs. when to ask? What signals indicate low confidence? Who gets notified, and how, when the agent escalates?

Design the off-ramp first.

🪵 Auditability

Can you reconstruct what the agent did and why? Every autonomous action should produce an artifact — a PR, a comment, a log entry — that a human can review after the fact.

If it can't be audited, it can't be trusted.

🔁 Feedback loops

How does the agent's behavior improve over time? Are bad autonomous decisions fed back into policy? Is the agent's track record measured?

Autonomous agents need performance reviews too.

Checklist

Use before sending any substantial prompt:

Context:

- ☐ Current state described in one sentence?
- ☐ Relevant file paths included?
- ☐ Related components or dependencies named?
- ☐ Prior session state summarized if continuing work?

Goal:

- ☐ Desired outcome stated as behavior?
- ☐ External docs or URLs linked?

Constraints:

- ☐ Backward compat requirements stated?
- ☐ Scope limits set (what *not* to touch)?
- ☐ Approach specified if it matters?

Verification:

- ☐ Success criteria or test command included?
- ☐ For complex tasks — using plan mode?
- ☐ Agent asked to verify before moving on?
- ☐ Pre-commit / linter / build check specified as done signal?

Curation:

- ☐ Only relevant files referenced (not the whole repo)?
- ☐ Error trimmed to the meaningful part?
- ☐ Unrelated background left out?
- ☐ AGENTS.md present and up-to-date for this project?
- ☐ Correct skill loaded if task is domain-specific?

Tool environment:

- ☐ Only MCP servers relevant to this task are loaded?
- ☐ Irrelevant integrations disabled for this session?

Key Takeaways

The agent can only act on what's in context — be the engineer writing the ticket

File paths and current state are not optional — specificity is the lever

Constraints prevent unwanted changes — say what not to touch

Feedback signals beat approval — test output > "looks good"

Context quality degrades as it grows — curate, don't dump

Too many tools pollutes the reasoning layer — load only what's relevant to the task

AGENTS.md and skills eliminate repetition — codify what you'd explain every session

Plan mode exists to catch mistakes before they happen — use it for risky changes

Each session starts fresh — re-anchor context, don't assume continuity

Autonomous agents need policy, not just capability — define authority boundaries before you build

These patterns are tool and model agnostic — they'll still work next year

Better prompts → fewer turns → faster results → more trust in the tool

Questions? Feedback?

What's working for you? What isn't? What did I get wrong?

🤝

Pairing Sessions

Happy to sit down and work through a real task together — sometimes the patterns click faster when applied to your own code.

🕐

Office Hours

If there's enough interest, I'll set up a recurring slot — bring prompts, bring problems, bring skepticism.

Slides at github.com/blarghmatey/presentations